IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

FPGA accelerator for LLM

Report of week 1

Work of :

Gauvrit Jean-Luc

As a intern student

With the supervision of :

Kim Tae-Wan

June 2025

Contents

List of Figures

List of Tables

1 Introduction

1.1 Presentation of the Weekly Report

This weekly report presents the progress of my work on designing and optimizing a hardware accelerator for Large Language Models (LLMs) using Field-Programmable Gate Arrays (FPGAs). Its objective is to document technical advancements, highlight open design questions, and ensure alignment with the research direction defined in collaboration with Professor Tae-Wan Kim. Each section outlines key investigations, experimental results, and technical challenges encountered during the week.

1.2 Presentation of the Subject

The goal of this project is to do inference with a LLM, specifically a variant of the DeepSeek-R1 family, on a FPGA platform. DeepSeek has recently demonstrated excellent inference performance across various benchmarks, offering competitive accuracy while being more lightweight and efficient than many other open-source LLMs of similar size. These characteristics make it a strong candidate for hardware-level optimization.

LLMs typically require considerable computational power and memory bandwidth, which limits their deployment in edge or low-power environments. While GPUs provide strong performance, their energy consumption is often prohibitive for embedded or mobile use cases.

This project explores the use of FPGAs to design a custom, energy-efficient accelerator optimized for DeepSeek inference. The flexibility of FPGAs allows for fine-tuned architectural choices targeting both compute and memory efficiency. Our focus is on reducing power consumption while maintaining acceptable latency (e.g., 100–200 ms per token), with attention given to constraints such as voltage levels, memory interfaces, and model size. The final goal is to assess whether FPGA-based deployment can offer a viable and scalable solution for running LLMs like DeepSeek in real-world, resource-constrained settings.

Figure 1: Logo de DeepSeek

Ultimately, the objective is to evaluate whether FPGA-based acceleration can provide a sustainable and scalable solution for running LLMs like DeepSeek in compact or embedded systems.

1.3 FPGA

Field-Programmable Gate Arrays (FPGAs) are reconfigurable integrated circuits that can be programmed to implement custom digital logic. Their main advantage lies in flexibility : unlike fixed-function hardware, FPGAs allow designers to create architectures tailored to specific applications, such as neural network inference. This adaptability makes them well suited for achieving an efficient balance between performance, power consumption, and resource usage.

2 Reminder

2.1 Transformer

2.1.1 Feedforward Neural Network

A feedforward neural network refers to a unidirectional architecture where information flows strictly from inputs to outputs through successive layers. The basic operation is a weighted sum of inputs followed by an activation function, and this pattern is repeated across layers.

At every stage of inference and training, matrix multiplication remains central, particularly during backpropagation. In this structure, there is no feedback or looping mechanism, hence the term “feedforward”.

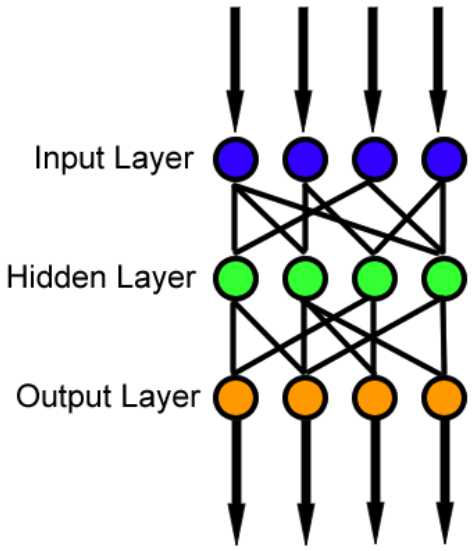

Figure 2: Feedforward network

As shown in Figure 2, data flows in one direction only ; no information is propagated backward or reused across time steps.

2.1.2 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are specifically designed for sequential data, where the order of elements is crucial, such as in text, speech, or time series. Unlike feedforward networks, RNNs include recurrent connections that allow information from previous steps to influence the current output.

Each RNN cell maintains a hidden state that acts as memory, updating at each time step based on the current input and the previous hidden state. This enables the model to capture temporal dependencies and context across sequences.

However, standard RNNs face limitations due to the vanishing gradient problem, which makes it difficult for them to retain long-term dependencies over extended sequences.

2.1.3 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are powerful architectures designed to process grid-like structured data such as images, audio, or text. They employ convolutional filters (kernels)

that slide across the input to detect localized patterns (e.g., edges or textures). Unlike fully connected networks, CNNs use shared weights in these filters, significantly reducing the number of parameters.

This structure enables hierarchical feature extraction, from simple features in early layers to more abstract representations in deeper layers making CNNs highly effective for tasks involving spatial and structural information.

2.2 The Rise of Transformers

The Transformer, a deep learning architecture introduced by Vaswani et al. in 2017 [4], has become the foundation of most modern LLMs. Initially developed for natural language processing (NLP), its application now extends to computer vision, audio processing, and even multimodal learning.

In contrast to RNNs, which process input sequentially, the Transformer utilizes a mechanism called self-attention to analyze all tokens in parallel. This parallelism greatly enhances training efficiency and scalability. Moreover, the model avoids the vanishing gradient problem by not relying on recursive state propagation.

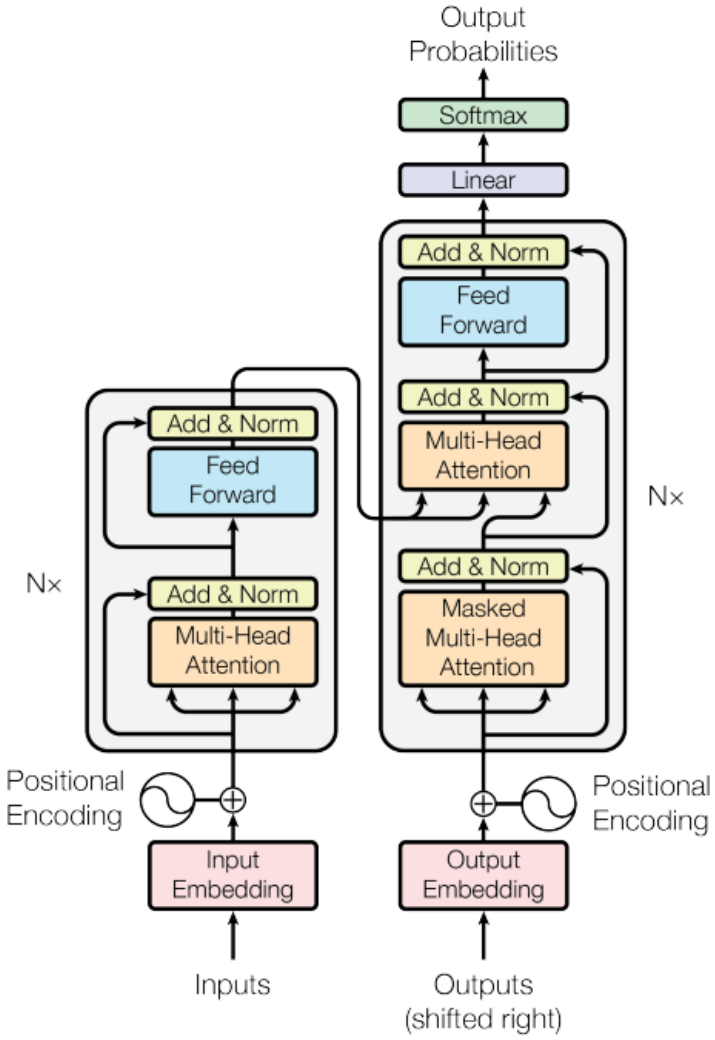

Figure 3: The Transformer model architecture [4]

The Transformer architecture is built from stacked encoder and decoder layers combining

multi-head self-attention and feedforward networks, enhanced by residual connections and normalization. The encoder encodes the input context, while the decoder generates the output — though modern LLMs like GPT typically use decoder-only variants for generation tasks.

At the core lies the attention mechanism, which computes the relevance between tokens using learned query, key, and value matrices. Multi-head attention allows the model to capture diverse relationships in parallel.

Thanks to its modular and parallel nature, the Transformer scales efficiently on large datasets with hardware accelerators. Our project targets FPGA-based inference acceleration by optimizing its two most compute-intensive operations: attention and feedforward layers.

2.3 Encoder-Decoder Architecture

2.3.1 Embedding

The first step in using a Large Language Model (LLM) is **embedding**. This process converts words (or other inputs) into numerical vectors that machine learning models can process. These vectors capture semantic information about words, enabling the model to understand relationships and meanings.

For example, using the pre-trained GloVe embedding model with 50-dimensional vectors, we can transform the word `"king"` into a numerical representation [1]:

```python
from gensim.downloader import load

model = load("glove-wiki-gigaword-50")  # Load GloVe model

king = model["king"]  # Get the vector for "king"

print("Vector of (king) :\n", king)
```

The result is:

```
Vector of (king) :
 [ 0.50451   0.68607  -0.59517  ...  -1.6106   -0.64426  -0.51042 ]
```

This 50-dimensional vector represents the word `"king"` in a way that reflects its meaning and usage. Words with similar meanings (e.g., `"queen"`, `"prince"`) will have vectors close to `"king"` in this high-dimensional space.

To demonstrate how embeddings capture semantic relationships, we can compute the scalar product (dot product) between word vectors. A higher scalar product indicates stronger similarity between two words.

[1] By comparison, GPT-3 uses embeddings with around 12,200 dimensions.

Here's a comparison between several word pairs :

```
import numpy as np

king = model["king"]
queen = model["queen"]
man = model["man"]
woman = model["woman"]
chicken = model["chicken"]
music = model["music"]

print("king · queen:", np.dot(king, queen))
print("man · woman:", np.dot(man, woman))
print("chicken · music:", np.dot(chicken, music))
```

Output :

```
king · queen: 21.877506
man · woman: 25.906174
chicken · music: 8.525226
```

This experiment highlights the ability of word embeddings to capture semantic relationships between words. Words with similar meanings or contexts, such as `"king"` and `"queen"` or `"man"` and `"woman"`, produce higher dot products, indicating strong vector alignment. In contrast, unrelated words like `"chicken"` and `"music"` yield much lower similarity scores. These results demonstrate how embedding spaces structure linguistic meaning in a way that can be exploited by models to perform reasoning, analogy completion, and contextual understanding.

2.3.2 Decoder Architecture

In the Transformer architecture, the **decoder** is responsible for generating the output sequence, token by token, based on the information encoded by the encoder. Each decoder layer is composed of three main components :

1. **Masked multi-head self-attention:** prevents the model from attending to future positions in the sequence, ensuring that predictions are made in an autoregressive manner.
2. **Encoder–decoder attention:** allows the decoder to focus on relevant parts of the encoder's output, aligning the generated output with the input sequence.
3. **Feedforward network:** processes the attention outputs through position-wise transformations to enhance representation capacity.

In practice, many modern LLMs such as GPT simplify this architecture by using only the decoder stack. These models are optimized for autoregressive generation, where the next token is predicted based solely on the previously generated tokens. In contrast, full encoder–decoder models—such as BART or T5—remain widely used for tasks that benefit from bidirectional context and complete input comprehension, such as translation, summarization, or question answering.

2.4 Multi-Head Attention

The core innovation of the Transformer architecture is its attention mechanism, and more specifically, **multi-head attention**. This mechanism allows the model to attend to different parts of the input sequence simultaneously, from different "perspectives" or representation subspaces.

In single-head attention, each token generates three vectors: a *query* Q, a *key* K, and a *value* V. These vectors are used to compute attention scores between tokens by measuring the similarity between queries and keys. The output is a weighted sum of values, emphasizing relevant information.

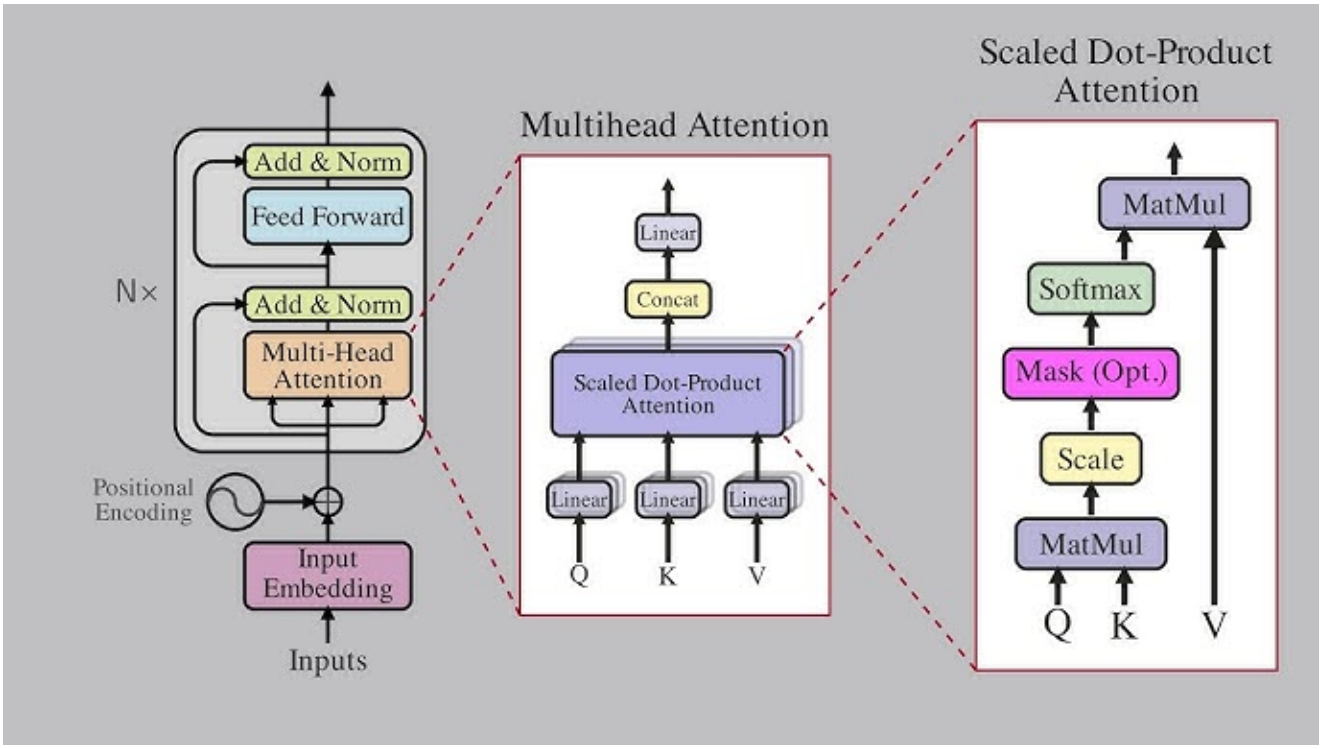

Figure 4: Multi-Head Attention

Multi-head attention extends this idea by creating several sets of Q, K, and V using different learned weight matrices. Each set (or "head") captures different types of relationships or dependencies. The outputs of all heads are then concatenated and projected, producing a richer and more nuanced representation.

This mechanism enables the model to:

- Focus on multiple parts of the sequence at once.
- Learn multiple types of dependencies (e.g., syntax, semantics).
- Improve generalization and expressiveness.

For hardware acceleration, multi-head attention presents a challenge due to its use of multiple parallel matrix multiplications and softmax operations, which must be carefully optimized for FPGA implementation.

2.5 Mathemmatics inside a LLM

2.5.1 Softmax

Softmax is a mathematical function that normalizes input scores into a probability distribution, making the outputs sum to 1. In attention mechanisms, it is used to compute attention weights, highlighting the relative importance of different elements in a sequence. Mathematically, it is defined as follows :

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{1}$$

where x_i represents the input scores.

2.5.2 ReLU (rectified linear unit)

ReLU is an activation function that introduces nonlinearity by allowing only positive values to pass through and setting negative values to zero. It is computationally efficient and helps prevent the vanishing gradient problem. The ReLU function is expressed as follows :

$$\text{ReLU}(x) = \max(0, x) \tag{2}$$

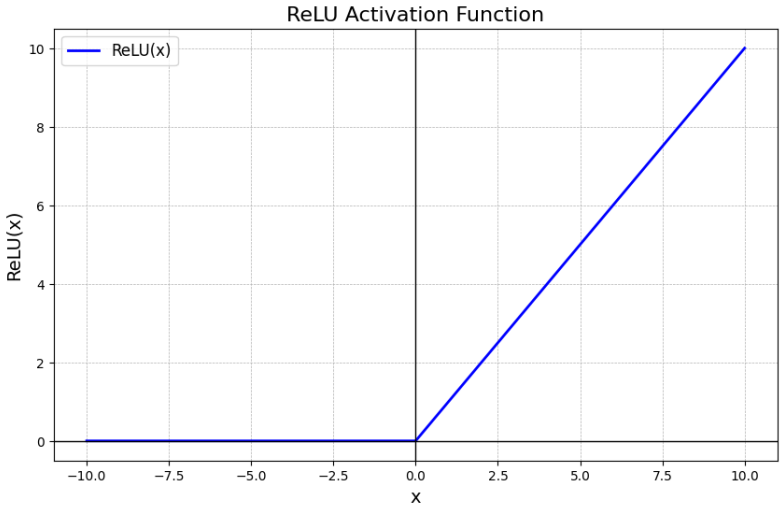

Figure 5: ReLU fonction

2.5.3 GELU (Gaussian Error Linear Unit)

GELU is a smooth, non-linear activation function that combines the benefits of ReLU and sigmoid functions. It enables models to capture nuanced patterns by scaling inputs based on their value. The GELU function is approximated as follows :

$$\text{GELU}(x) = x \cdot \Phi(x) \tag{3}$$

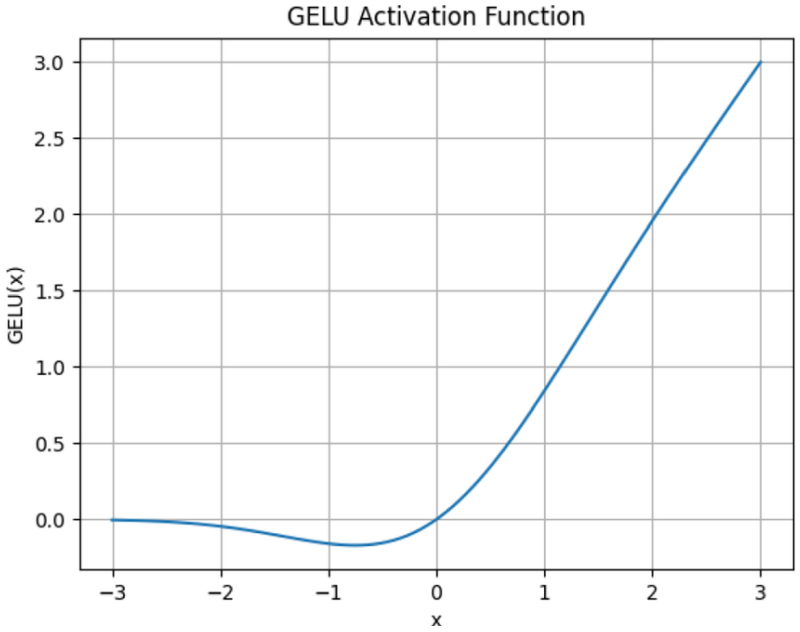

Figure 6: GELU fonction

where $\Phi(x)$ is the cumulative distribution function of a standard normal distribution. GELU is commonly used in Transformers as it provides better gradient flow and performance compared to ReLU.

An approximation of $\Phi(x)$ is often used in practice, as follows :

$$\text{GELU}(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715 \cdot x^3)\right)\right) \tag{4}$$

These building blocks work in tandem to enable attention networks to model complex relationships within data, contributing to their effectiveness in natural language processing and other tasks. Together with GEMM, these building blocks are often the most computationally intensive parts of the LLM that are often mapped to specialized hardware resources.

3 Exploration of Existing Solutions

3.1 Analysis of the article from Christoforos Kachris [2]

In January 2025, Christoforos Kachris published a comprehensive survey entitled *"A Survey on Hardware Accelerators for Large Language Models"* [2]. He provides a comparative analysis of hardware platforms for accelerating LLMs, including GPUs, FPGAs, ASICs, and in-memory computing. The paper discusses the trade-offs between performance, energy efficiency, flexibility, and scalability.

This survey is directly relevant to our work, as it highlights the potential of FPGAs for designing energy-efficient and reconfigurable accelerators suited to LLM inference in edge computing environments.

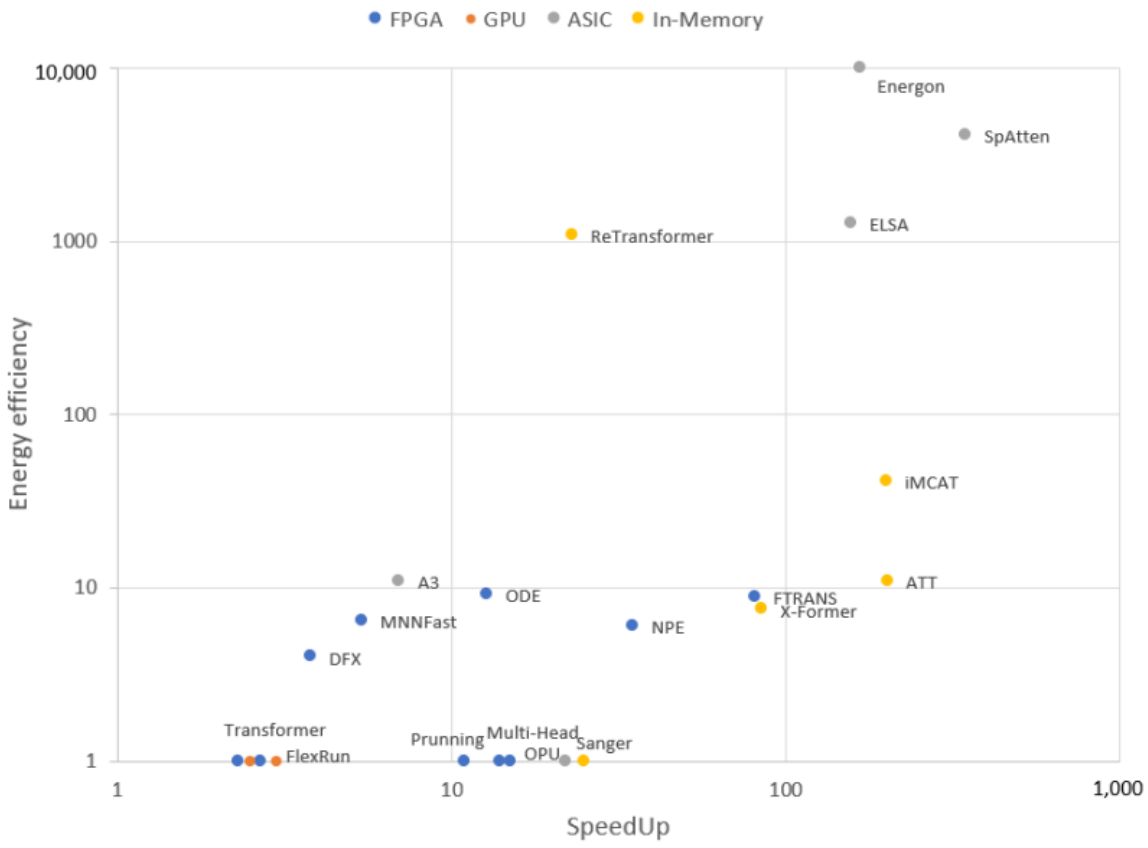

Figure 7: Speedup versus energy efficiency [2]

Figure 7 presents a scatter plot of various hardware acceleration technologies, showing their positioning in terms of energy efficiency versus speedup.

3.2 FPGA-Based Accelerators

3.2.1 MNNFast

MNNFast accelerates large-scale language models through three key optimizations [1] :

1. **Reduced memory bandwidth consumption** : MNNFast introduces a column-based computation algorithm with streaming support, which minimizes the size of data spills and hides most off-chip memory access latency.

2. **Zero-skipping optimization** : It avoids unnecessary output computations by skipping operations associated with values close to zero in the softmax probability vector.

3. **Dedicated embedding cache** : To address cache contention between embedding and inference operations, MNNFast implements a specialized embedding cache that isolates embedding matrix accesses from other memory traffic.

For our project, we are particularly interested in the third optimization : the embedding cache. This approach can be combined with other FPGA-based acceleration techniques. Moreover, MNNFast is a technology developed at Seoul National University, which strengthens its relevance and accessibility within the scope of our collaboration.

3.2.2 FTRANS

FTRANS combines two main strategies [3] :

1. **Enhanced BCM-Based Compression** : FTRANS replaces dense weight matrices in Transformer layers (e.g., attention and feed-forward weights) with block-circulant matrices (BCMs). This compression reduces the model size by up to 16× with minimal accuracy degradation. The technique uses an improved formulation for circulant indexing, preserving more information than previous approaches.

2. **FPGA Architecture Optimization** : The framework implements an optimized hardware design, separating the embedding layer (stored off-chip) from the encoder-decoder blocks (placed on-chip). It uses pipelining, scheduling, and modular PE (Processing Element) units to maximize throughput while minimizing power usage.

Experimental results show that FTRANS achieves up to 8.80× higher energy efficiency compared to GPU implementations, and more than 80× compared to CPU. It supports state-of-the-art Transformer model, as well as smaller and shallower variants, offering flexibility for scaling across different LLM sizes and configurations.

It is the most hopeful technologies for our project.

3.2.3 FPGA DFX (Simplified Edge-Oriented View)

Unlike traditional batch-based designs, DFX is optimized for **single-token processing**, which is essential for reducing latency during sequential inference.

Although originally deployed across **multiple high-end FPGAs** with HBM (e.g., Xilinx Alveo U280), DFX introduces architectural ideas that are valuable for edge applications :

- **Modular design** with a Matrix Processing Unit (MPU) and Vector Processing Unit (VPU), which can be scaled or simplified depending on the FPGA size.

- **Data tiling and custom dataflow**, which improves memory access efficiency — crucial when using lower bandwidth memory (e.g., LPDDR4).

- **Instruction-level control**, allowing low-latency, token-by-token inference with fine-grained scheduling.

While the original implementation targets data centers, the concepts in DFX can inspire a lightweight version adapted to smaller FPGA platforms. Its focus on latency and modularity aligns well with edge deployment goals, especially for models like DeepSeek where token-level control is critical.

3.3 Other Strategies

3.3.1 ASIC Implementations

ASIC accelerators provide the best trade-off between performance and energy efficiency. As shown in Figure 7, designs like *SpAtten* and *ELSA*, both built on 40nm technology.

Among them, **Energon** stands out for its use of a mixed-precision multi-round filtering (MP-MRF) algorithm. This technique applies high-precision attention only to the most relevant token pairs, reducing computation by 4× to 8× with minimal accuracy loss. While ASICs are not deployable on our platform, Energon's sparse attention strategy is promising. Adapting it to FPGA could significantly lower power consumption, in line with our goal of efficient DeepSeek inference.

3.3.2 Scalability and Flexibility

To support broader deployment scenarios and future scaling needs, it is important to consider the scalability of our hardware architecture. While this project initially targets edge-level inference, larger use cases may require a system capable of handling more demanding workloads or supporting multiple LLM variants.

In this context, AMD's FPGA product line—particularly the Xilinx Alveo series—offers promising solutions. These data center-grade FPGA cards provide a balance of performance, memory bandwidth, and reconfigurability. They are designed to scale across applications, supporting high-throughput workloads while remaining power-efficient.

The documentation for the Xilinx Alveo platform highlights its flexibility for accelerating AI inference, including Transformer-based models. This makes it a valuable candidate for future iterations of the project, should we transition from low-power edge devices to larger or cloud-based deployments.

4 Technical choice

4.1 The DeepSeek version

In the context of LLM acceleration, the underlying architecture plays a crucial role in determining the feasibility of FPGA deployment. Selecting a model with manageable inference complexity and high scalability is essential to meet power and latency constraints.

Aspect	**DeepSeek R1-Zero**	**DeepSeek R1**
Primary Goal	Proof of concept for pure RL reasoning; research focus.	Optimized reasoning model for practical applications.
Architectural Base	DeepSeek-V3-Base (MoE, 671B parameters, 37B activated per token).	DeepSeek-V3-Base (MoE, 671B parameters, 37B activated per token).
Emergent Reasoning	Yes, sophisticated CoT behaviors emerged via RL.	Yes, builds upon R1-Zero's emergent capabilities.
Readability & Coherence	Poor; fragmented, unclear explanations, inconsistent formatting.	Significantly improved; clearer, more structured, more polished responses.
Language Mixing	Frequent (English/Chinese in responses).	Largely resolved and reduced.
Output Quality	Logically correct but less polished, reduced usability.	Highly polished, coherent, and user-friendly.
Average Tokens (ARC-AGI-1)	11K.	6K.
Average Cost (ARC-AGI-1)	$0.11	$0.06
Primary Use Cases	Pure RL reasoning research, foundational studies.	Complex mathematics, coding, scientific reasoning, multi-step planning, general Q&A.
Relationship	Precursor/Base model for R1.	Refined, production-ready version, building upon R1-Zero.
Latest Updates	N/A (foundation, no specific version updates mentioned).	R1-0528 (stronger reasoning, reduced hallucination, function calling, system prompt support).

Table 1: Comparison between DeepSeek R1-Zero and DeepSeek R1

Given these considerations, I chose to work with **DeepSeek R1**. Its improved coherence, reduced complexity in inference, and enhanced scalability. Particularly in the R1-0528 version, make it a more suitable candidate for energy-efficient deployment on FPGA.

While the Distill versions are generally more challenging to scale efficiently, it would still be interesting to run tests with the DeepSeek R1 Distill Qwen 1.5B model as a lightweight baseline for comparison.

4.2 PC configuration

As a first step, the inference will be tested on a high-performance personal computer before porting the model to FPGA. The target configuration is as follows :

- **Operating System:** Ubuntu (latest version)
- **Memory:** 64 GB DDR4 or DDR5 RAM
- **Processor:** AMD[2] CPU with at least 16 cores and a base clock of 3.5 GHz or higher
- **GPU:** NVIDIA RTX 4090

This setup will allow full-scale testing of DeepSeek inference under optimal conditions and serve as a baseline for performance and energy comparison with the future FPGA implementation.

4.3 FPGA plateform

This kind of FPGA plateform is very expensive.
AMD Virtex™ UltraScale+™ FPGA VCU118 Evaluation Kit
AMD Zynq UltraScale+™ MPSoC ZCU102 Evaluation Kit

[2]Choosing an AMD processor ensures consistency in the instruction set architecture, which can simplify inference debugging and reproducibility across test environments.

5 Conclusion

The above strategies can be mixed and matched depending on hardware availability, model size, and latency/power constraints. All selected designs are compatible with Vivado and target Xilinx FPGA platforms, ensuring a unified development environment and long-term scalability. This modular approach allows the system to evolve from DeepSeek-R1 (1.5B) to larger variants (up to 8B) by gradually integrating more FPGA resources or optimizing the pipeline.

References

[1] Hanhwi Jang, Joonsung Kim, Jae-Eon Jo, Jaewon Lee, and Jangwoo Kim. Mnnfast: a fast and scalable system architecture for memory-augmented neural networks. In *Proceedings of the 46th International Symposium on Computer Architecture*, ISCA '19, page 250–263, New York, NY, USA, 2019. Association for Computing Machinery.

[2] Christoforos Kachris. A survey on hardware accelerators for large language models. *Applied Sciences*, 15(2):586, January 2025.

[3] Bingbing Li, Santosh Pandey, Haowen Fang, Yanjun Lyv, Ji Li, Jieyang Chen, Mimi Xie, Lipeng Wan, Hang Liu, and Caiwen Ding. Ftrans: Energy-efficient acceleration of transformers using fpga, 2020.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.